

Sistemas Operativos
3 de febrero de 2021, Curso 2020-21
Facultad de Informática, UCM

Cuestión 1. Un usuario desea dar formato a una partición de disco para almacenar su colección de fotografías. Cada fotografía se almacena en un fichero, con un tamaño (constante) de 7000 bytes.

El sistema de ficheros elegido está basado en i-nodos; la estructura de cada inodo está formada por 2 enlaces directos y 1 indirecto simple. Además, se usan 4 bytes para identificar un inodo, siendo el tamaño de puntero a bloque de 32 bits.

En el momento de proceder con el formateado del disco, se le ofrecen tres opciones distintas como tamaño de bloque a utilizar: 1024 bytes, 2048 bytes y 4096 bytes.

Ayude al usuario a tomar una decisión, contestando razonadamente a las siguientes preguntas:

- a) ¿Cuál de las tres opciones presenta menos fragmentación interna? Para cada tamaño de bloque, indique el porcentaje de ocupación real de cada uno de los bloques de datos asignados a un fichero.
- b) ¿Cuál de las tres opciones ofrece un mejor aprovechamiento de los bloques de disco disponibles? Para cada tamaño de bloque, indique qué porcentaje de bloques de disco se utilizarán para almacenar los datos de un determinado fichero.
- c) ¿Cuál de las tres opciones ofrecerá, teóricamente, un mejor rendimiento de entrada/salida para la transferencia de un bloque de datos individual?
- d) ¿Cuántos accesos a disco requerirá una lectura secuencial completa de una fotografía en cada caso, suponiendo que todas ellas se almacenan en un único directorio con nombre `fotografias` cuyas entradas se almacenan en un solo bloque de datos, y que la partición se montará en el directorio raíz (/) del sistema?
- e) ¿Cómo afectará la selección de un tamaño de bloque determinado al tamaño de la tabla de inodos y al mapa de bits de bloques libres?

Cuestión 2. Se desea simular el trabajo en un muelle de carga, usando hilos para simular los operarios que realizan la carga y los camiones que son cargados. El siguiente código describe la estructura de los hilos utilizados para los trabajadores y los camiones:

```
#define NP 10 // Capacidad máxima de cada camión

void *worker(void *arg)
{
    while (1) {
        // esperar a poder entrar

        enter_dock();

        // esperar a que el camión esté cargado

        delivery();
    }
}

void *truck(void *arg)
{
    while (1) {
        // esperar a poder cargar

        load_parcel(p);

        // notificar si cargado
    }
}
```

Los operarios (`worker`) repiten indefinidamente las acciones de coger un paquete (`get_parcel`) y cargarlo en el camión (`load_parcel`). Varios operarios pueden estar cogiendo un paquete al mismo tiempo, pero cargarán el camión de uno en uno. El hilo del operario sólo debe ejecutar la función `load_parcel` cuando haya algún camión en el muelle y quede sitio en dicho camión para el paquete.

Los camiones (`truck`) tiene capacidad para cargar `NP` paquetes, y repiten indefinidamente las acciones de entrar al muelle (`enter_dock`) para recoger la carga y salir al reparto (`delivery`). Varios hilos de camión pueden estar ejecutando la función `delivery` al mismo tiempo. Sin embargo, en el muelle sólo cabe un camión y por tanto el hilo que simula al camión debe esperar a que el muelle quede libre para ejecutar la función `enter_dock` en exclusiva. Asimismo, el hilo debe esperar a que el camión esté completamente cargado para salir del muelle y empezar el reparto.

Completa las funciones `truck` y `worker` para que los hilos se sincronicen correctamente, añadiendo las variables que consideres necesarias.

Solución:

```
#define N 10
#define M 3
#define NP 10

mutex_t mtx;
cond_t space, loaded, dempty;
int dock_empty = 1;
int truck_space;

void *truck(void *arg)
{
    int id = (int) arg;

    while (1) {
        mutex_lock(&mtx);
        while (!dock_empty)
            cond_wait(&dempty, &mtx);
        truck_space = NP;
        dock_empty = 0;
        cond_broadcast(&space);

        enter_dock(id);

        while (truck_space)
            cond_wait(&loaded, &mtx);
        dock_empty = 1;
        cond_signal(&dempty);
        mutex_unlock(&mtx);

        delivery();
    }
}

void *worker(void *arg)
{
    int id = (int) arg;
    int p;

    while (1) {
        p = get_parcel();

        lock(&mtx);
        while (dock_empty || truck_space < 1)
            cond_wait(&space, &mtx);
        truck_space--;

        load_parcel(id, p);

        if (truck_space == 0)
            cond_broadcast(&loaded);
        unlock(&mtx);
    }
}
```

Cuestión 3. Un SO aplica una política de planificación con colas multinivel (3 niveles) y realimentación a las tareas siguiendo las siguientes directrices:

- Las tareas nuevas entran al nivel 1.
- En todos los niveles se aplica planificación **Round-Robin**, con quantos crecientes: q=2 (nivel1), q=3 (nivel2), q=4 (nivel3).
- Las tareas del nivel 1 que durante una ráfaga de CPU agoten su cuanto y necesiten más CPU pasarán al nivel 2. Si la tarea finaliza su ráfaga justo al agotar su cuanto permanece en el nivel.
- Las tareas del nivel 2 que durante una ráfaga de CPU agoten su cuanto y necesiten más CPU pasarán al nivel 3. Si la tarea finaliza su ráfaga justo al agotar su cuanto permanece en el nivel.
- Cada 15 unidades de tiempo, comenzando en el instante 0, (instantes 15,30,...) todos los procesos que se encuentren en el tercer nivel, o estén bloqueados por entrada salida pero fueron planificados por última vez desde este nivel, recibirán una actualización de su prioridad y pasarán a la cola de nivel 1. Esta mejora es independiente del instante del instante de llega a dicho nivel. Si hay varias tareas que mejoran su prioridad, el orden que tuvieran entre ellas en el nivel 3 se conserva en el nivel 1.
- Las tareas que retornen de entrada salida se mantendrán en el nivel en el que fueron planificadas por última vez, independientemente del nivel.

A) Identificas problemas que puede presentar este algoritmo de planificación?. En caso afirmativo, describirlos razonando la respuesta.

B) Aplíquelo a las siguientes trazas:

Tarea	Llegada	CPU	E/S	CPU	E/S	CPU
T1	0	1	3	1	4	3
T2	2	7	1			
T3	1	3	1	1	1	1
T4	3	5	1	1		

Completar el diagrama temporal, indicando también los procesos preparados que hay en cada una de las colas. Al rellenar el diagrama temporal, se ha de usar el siguiente convenio:

- En ejecución: marcar con X
- Bloqueado por E/S: marcar con O
- Listo para ejecutar: marcar con –

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23
T1																								
T2																								
T3																								
T4																								
Q1																								
Q2																								
Q3																								

Cuestión 4. Un sistema utiliza gestión de memoria virtual con paginación bajo demanda con páginas de 2KiB, direcciones de memoria de 32 bits y memoria principal de 16GB. En dicho sistema se desea ejecutar el siguiente programa:

```
#include <stdio.h>
/* resto de includes */
...

#define N 1024
int A[N] = {0};
int *B;

void* copyv(void* arg);

int main(int argc, char* argv[])
{
    int pid,n,i,fd;
    pthread_t th1, th2;
    int arg1[2], arg2[2];

    for (i=0; i<N; i++)
        A[i] = i+1;
    printf("Initialization was OK");
    B=malloc(N*sizeof(int))
    arg1[0] = 0;   arg1[1] = N/2 - 1;
    arg2[0] = N/2; arg2[1] = N-1;
```

```
    pthread_create(&th1, NULL,
        copyv, (void*) arg1);
    pthread_create(&th2, NULL,
        copyv, (void*) arg2);

    pthread_join(th1, NULL);
    pthread_join(th2, NULL);

    for (i=0; i<N; i++)
        printf("B[%i]=%i\n",i,B[i]);

    return 0;
}

void* copyv(void* arg)
{
    int* params = (int*) arg;
    int j;
    for (j=params[0]; j <= params[1]; j++)
        B[j] = A[j];

    return NULL;
}
```

Considerando que:

- Las llamadas al sistema y el resto de funciones de librería que se incluyen en el programa nunca fallan.
- Todas las funciones externas se incluyen en una única biblioteca estática.
- El texto (código) del programa tras la compilación y enlazado estático ocupa 103KiB.
- Un entero en C ocupa 4 bytes.

Responda a las siguientes preguntas:

- Para las siguientes variables, macros y código de las funciones usadas en el programa C indique cuáles ocupan espacio en el fichero ejecutable de dicho programa, y en qué sección del ejecutable se encuentran. Indique también la región del mapa de memoria en la que se emplazan, así como el tamaño máximo en páginas que pueden ocupar dichas regiones durante la ejecución del programa.

Símbolo o expresión	Ocupa espacio en el ejecutable (Sí/No) - Sección del ejecutable	Región del mapa de memoria - Tamaño máximo en Páginas
N		
pid		
A		
main()		
j		
B		
malloc()		

- b. El programa anterior hace uso de múltiples hilos para realizar cooperativamente la copia de un vector en otro. Indique si sería posible realizar esta copia utilizando múltiples procesos que cooperasen mediante el uso de alguna abstracción o servicio proporcionado por el sistema operativo. En caso afirmativo, describa a alto nivel una posible implementación, y discuta qué diferencias existirían entre el proceso multihilo anterior y la aplicación multiproceso en cuanto a la organización del mapa de memoria (qué regiones nuevas aparecerán y cuáles no estarán presentes).

Cuestión 5. Un proceso en UNIX ejecuta el siguiente código:

```
1  #include <fcntl.h>
2  #include <stdio.h>
3  #include <stdlib.h>
4  #include <sys/types.h>
5  #include <unistd.h>
6  #include <sys/wait.h>
7  #include <pthread.h>
8
9  #define N 5
10
11  int *fibseq;
12  char message[] = "Hello Fibonacci";
13
14
15  void fibo (int id) {
16
17      if(id==0)
18          fibseq[id] = 0;
19      else if (i==1)
20          fibseq[id] = 1;
21      else
22          fibseq[id] = fibseq[id - 1] + fibseq[id - 2];
23
24      printf ("fib %d: %d\n",id,fibseq[id]);
25  }
26
27
28
29  int main(){
30
31      int i;
32      pid_t pid;
33
34      fibseq = (int *)malloc(N * sizeof(int));
35
36      for(i=0;i<N;i++)
37          fibseq[i]=-1;
38
39      for (i = 0; i < N; i++) {
40          pid = fork();
41          if (pid < 0) {
42              perror("fork");
43              exit(EXIT_FAILURE);
44          } else if (pid == 0) {
45              fibo(i);
46              exit(EXIT_FAILURE);
47          }
48      }
49      while(execlp("/bin/echo", "/bin/echo",
50                  message, NULL) > 0);
51
52      printf ("pid= %d ppid= %d id %d= %d\n",
53              getpid(), getppid(), N, fibseq[N-1]);
54
55      exit(EXIT_SUCCESS);
56  }
```

Asumiendo que el proceso original tiene PID 100 (y es hijo de *Init*), se pide:

- Explica razonadamente cuántos procesos serán creados y cuántos ejecutarán potencialmente de forma concurrente.
- Justifica si se puede determinar o no el orden en que se ejecutarán estos procesos, indicando en tal caso dicho orden. En cualquier caso, indica una posible salida del código anterior.
- Indica una posible salida si cambiamos la función `execlp (" . . . ")` por `wait (NULL)`.